UNIVERSITY OF GEORGIA

CSCI/ARTI 8950 Machine Learning

Lecture Notes 3: Decision Trees

Khaled Rasheed
Professor
School of Computing
Member
Institute for Artificial Intelligence
University of Georgia

Credit: some examples & figures by Marine Carpuat, Tom Mitchell and Guoping Qiu

Machine Learning as Function Approximation

Problem setting

- Set of possible instances X
- Unknown target function $f: X \rightarrow Y$
- Set of function hypotheses $H = \{h \mid h: X \rightarrow Y\}$

Input

- Training examples $\{(x^{(1)}, y^{(1)}), \ldots (x^{(N)}, y^{(N)})\}$ of unknown target function f

Output

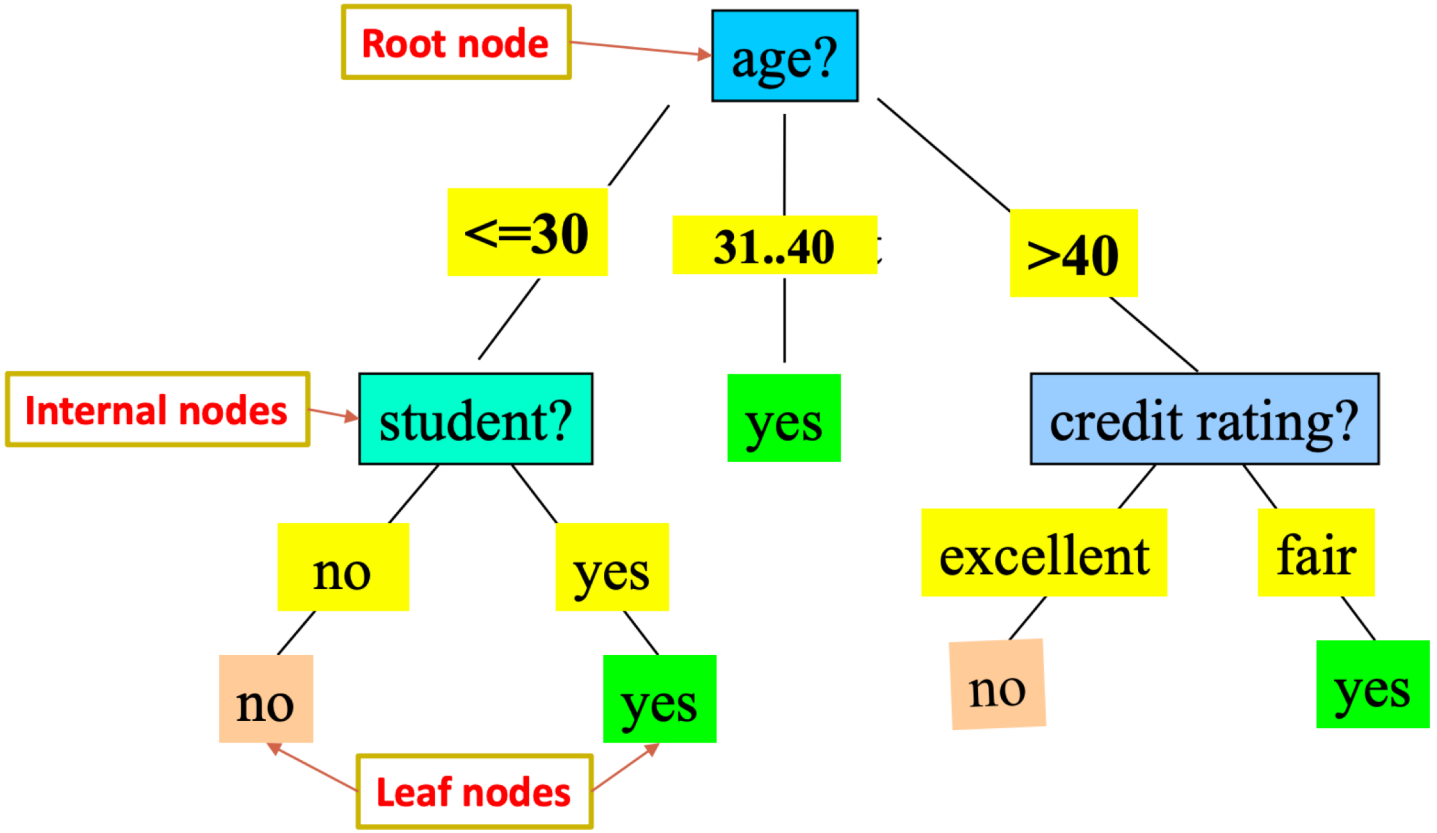
- Hypothesis $h \in H$ that best approximates target function f

Today: Decision Trees

- **What is a decision tree?**
- How to learn a decision tree from data?

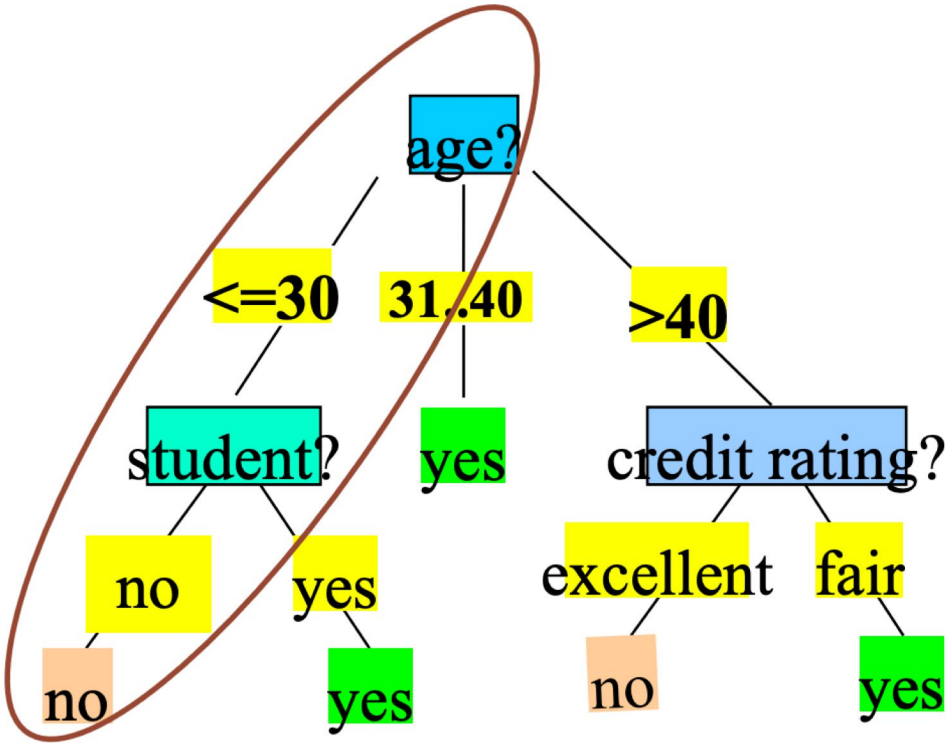
Tree-based Models

- Use trees to partition the data into different regions and make predictions



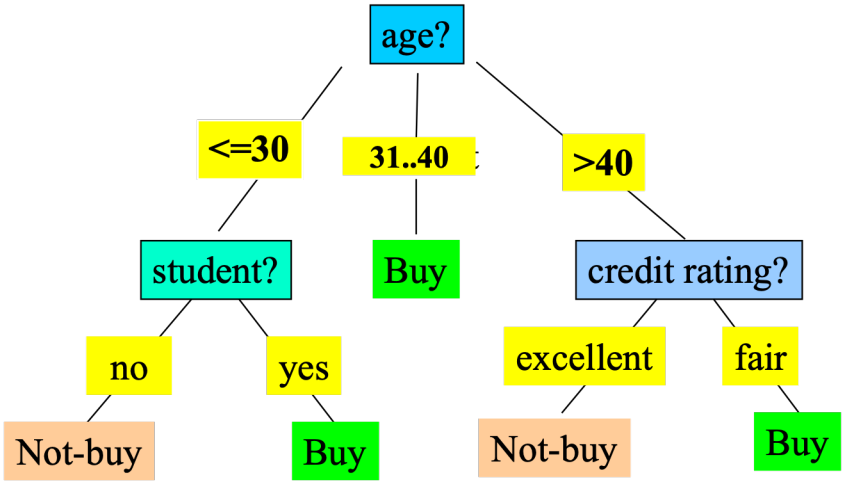
Easy to Interpret

- A path from root to a leaf node corresponds to a rule
  - E.g., if age<=30 and student=no then target value=no



Decision Tree Induction

- **Decision tree construction**:
  - A top-down, recursive, divide-and-conquer process
- **Resulting tree:**



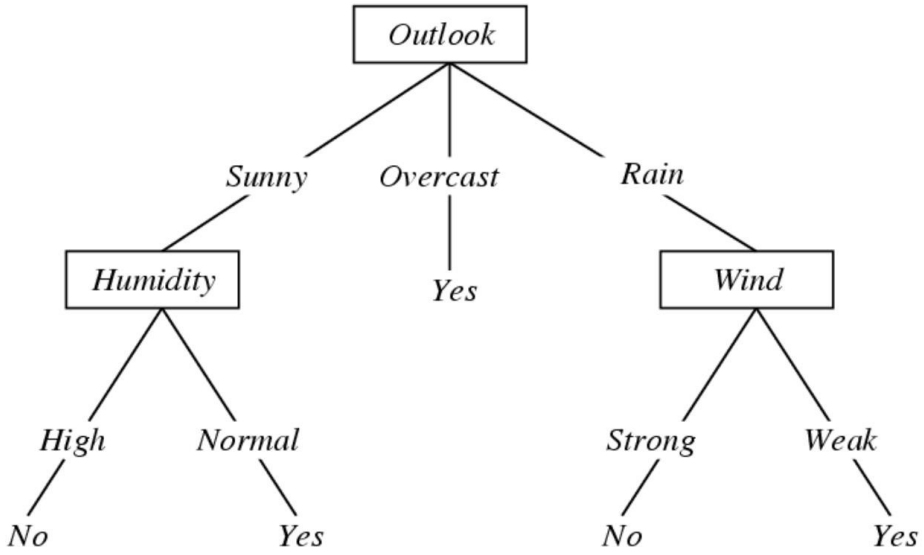
age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Training data set: Who buys computer?

An example training set

Day	Outlook	Temperature	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

A decision tree to decide whether to play tennis



Decision Trees

- Representation
  - Each internal node tests a feature
  - Each branch corresponds to a feature value
  - Each leaf node assigns a classification

- Decision trees represent functions that map examples in X to classes in Y

f: <Outlook, Temperature, Humidity, Wind> → PlayTennis?

Exercise

- How would you represent the following Boolean functions with decision trees?
  - AND
  - OR
  - XOR
  - $(A \cap B) \cup (C \cap \neg D)$

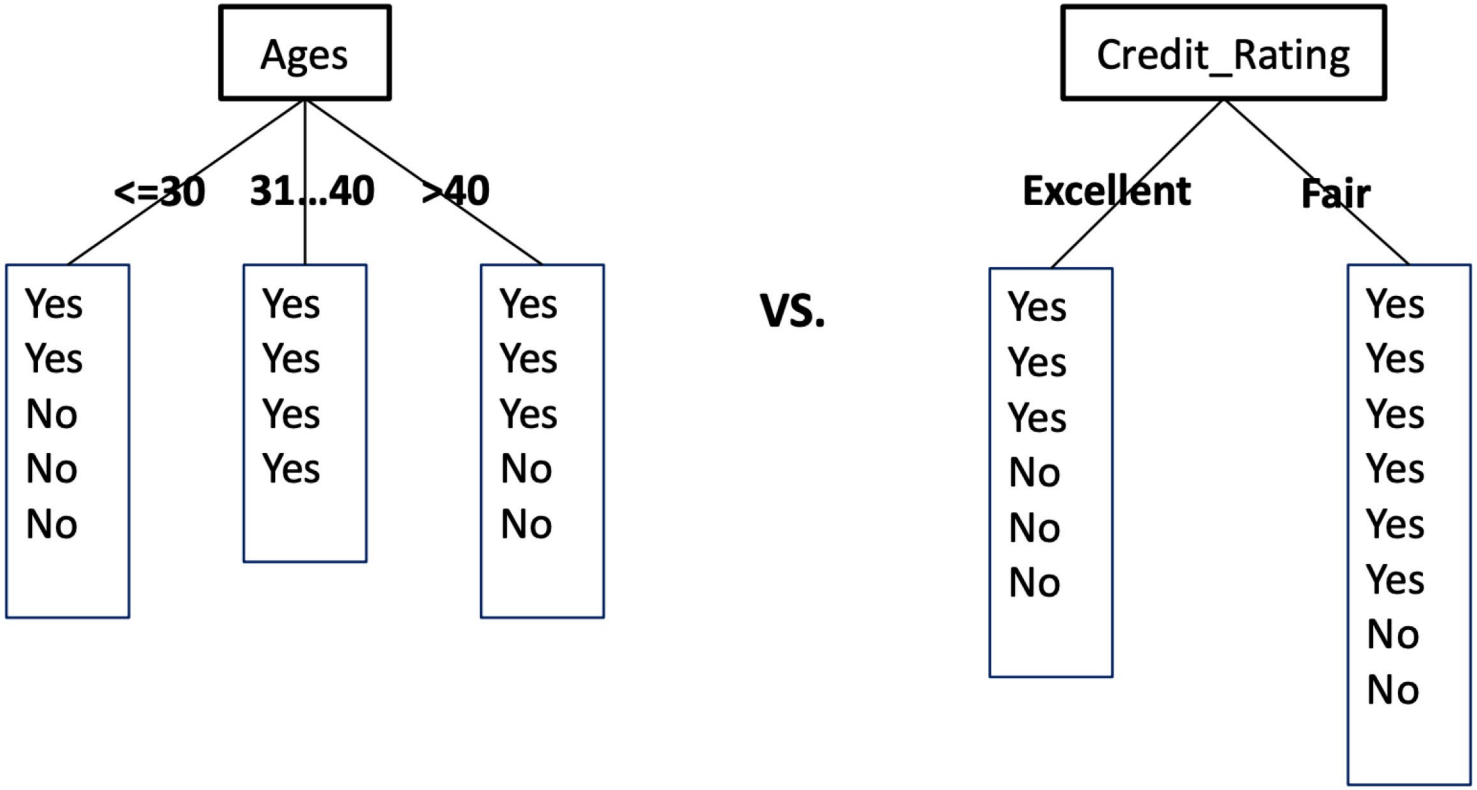
When to use a Decision Tree?

- Examples are represented by a small to moderate number of attributes
- Each attribute conveys significant information
  - Not suitable for sensory data (e.g. Pixels of an image)
- The label (class) must be discrete
  - Extensions for continuous prediction include Model Trees and Regression Trees

Today: Decision Trees

- What is a decision tree?
- **How to learn a decision tree from data?**

How to Choose Attribute?



Q: Which attribute is better for the classification task?

Function Approximation with Decision Trees

Problem setting

- Set of possible instances X
  - Each instance $x \in X$ is a feature vector $x = [x_1, \ldots, x_D]$
- Unknown target function $f: X \rightarrow Y$
  - Y is discrete valued
- Set of function hypotheses $H = \{h \mid h: X \rightarrow Y\}$
  - Each hypothesis h is a decision tree

Input

- Training examples $\{(x^{(1)}, y^{(1)}), \ldots (x^{(N)}, y^{(N)})\}$ of unknown target function f

Output

- Hypothesis $h \in H$ that best approximates target function f

Decision Tree Learning

- Finding the hypothesis $h \in H$
  - That minimizes training error
  - Or maximizes training accuracy

- How?
  - H is too large for exhaustive search!
  - We will use a **heuristic search** algorithm
    - Pick questions to ask, in order
    - Such that classification accuracy is maximized

Top-down Induction of Decision Trees

```
CurrentNode = Root

DTtrain(examples for CurrentNode,features at CurrentNode):
    1. Find F, the “best” decision feature for next node
    2. For each value of F, create new descendant of node
    3. Sort training examples to leaf nodes
    4. If training examples perfectly classified at leaf
         Stop
       Else
         Recursively apply DTtrain over new leaf nodes
```

How to select the “best” feature?

- A good feature is a feature that lets us make correct classification decision
- Criteria to measure how good a feature is
  - Classification accuracy
  - Information Gain (using entropy)
  - The GINI Index (used in CART, IBM IntelligentMiner)
    - https://www.slideshare.net/amrsb/decision-trees-for-machine-learning
  - …
- Let’s try some on the PlayTennis dataset

Let's build a decision tree using features W, H, T

Day	Outlook	Temperature	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Partitioning examples according to Humidity feature

Day	Outlook	Temperature	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Partitioning examples: Humidity = High

Day	Outlook	Temperature	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Majority label=No, Error=3 if we make a leaf and stop

Partitioning examples: Humidity= Normal

Day	Outlook	Temperature	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Majority label=Yes, Error=1 if we make a leaf and stop

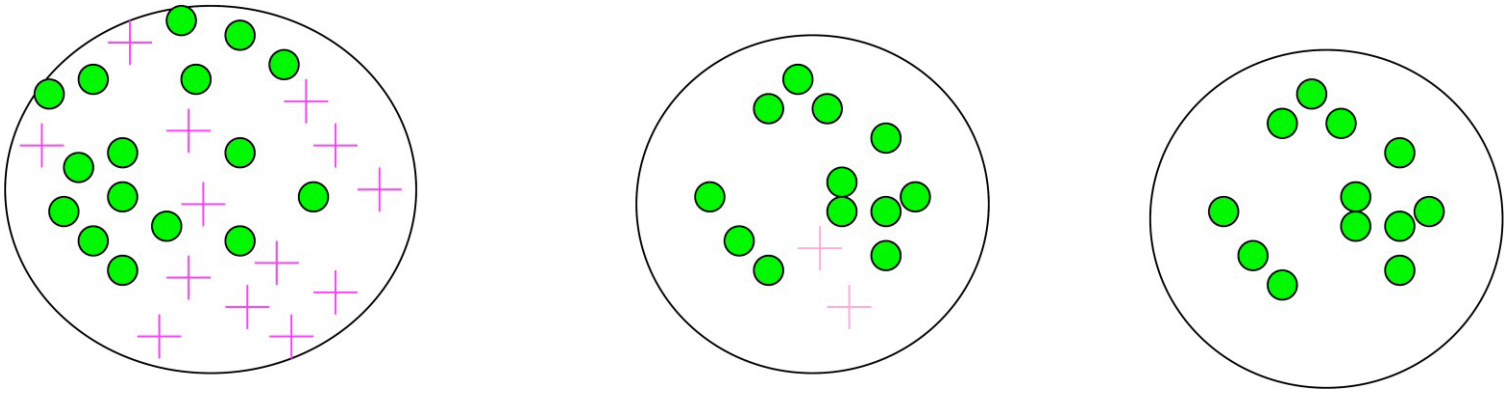
Can we do better?
H = Normal and W = Strong

Day	Outlook	Temperature	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Another feature selection criterion: Entropy

- Used in the ID3 algorithm [Quinlan]
  - pick feature with smallest entropy to split the examples at current iteration

- Entropy measures impurity of a sample of examples

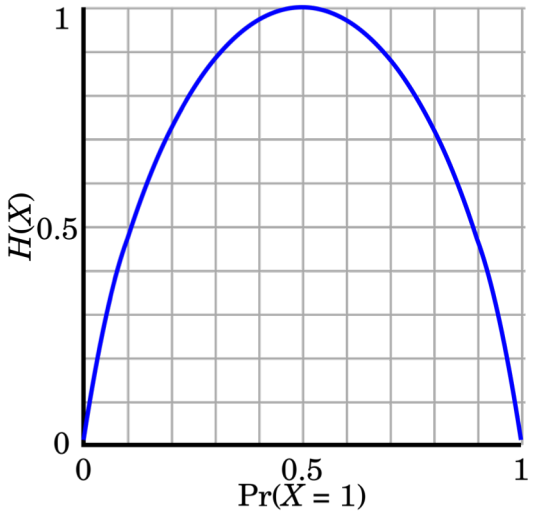


Entropy

- **Entropy (Information Theory)**
  - A measure of uncertainty associated with a random number
  - Calculation: For a discrete random variable Y taking m distinct values $\{y_1, y_2, \ldots, y_m\}$

$$H(Y) = -\sum_{i=1}^{m} p_i \log(p_i) \quad where\ p_i = P(Y = y_i)$$

  - Interpretation
    - Higher entropy → higher uncertainty
    - Lower entropy → lower uncertainty



m = 2

- **Conditional entropy**

$$H(Y|X) = \sum_{x} p(x) H(Y|X = x)$$

Information Gain

- Select the attribute with the highest information gain (used in classical algorithms: ID3/C4.5/C5)
- Let p_i be the probability that an arbitrary tuple in D belongs to class C_i estimated by $|C_i, D| / |D|$
- Expected information (entropy) needed to classify a tuple in D:

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

- Information needed (after using A to split D into v partitions) to classify D :

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

- Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

Information Gain: Example

- Class P: buys_computer = "yes"
- Class N: buys_computer = "no"

age	p_i	n_i	$I(p_i, n_i)$
<=30	2	3	0.971
31...40	4	0	0
>40	3	2	0.971

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Step 1: Calculate $Info(D)$

$$Info(D) = I(9,5) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940$$

Step 2: Calculate $Info_A(D)$ for each feature A

$$Info_{age}(D) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.694$$

$\frac{5}{14} I(2,3)$ means "age <=30" has 5 out of 14 samples, with 2 yes's and 3 no's. Hence

$$Gain(age) = Info(D) - Info_{age}(D) = 0.246$$

Similarly,

$$Gain(income) = 0.029$$
$$Gain(student) = 0.151$$
$$Gain(credit\_rating) = 0.048$$

Step 3: Choose the feature with highest Gain (e.g., age in this example)

Continuous Attributes

- Real-valued attributes can, in advance, be discretized into ranges, such as *big*, *medium*, *small*
- Alternatively, one can develop splitting nodes based on thresholds of the form $A < c$ that partition the data into examples that satisfy $A < c$ and $A >= c$. The information gain for these splits is calculated in the same way and compared to the information gain of discrete splits.

How to find the split with the highest gain ?

- For each continuous feature *A:*
  - Sort examples according to the value of *A*
  - For each ordered pair (x, y) with different labels
    - Check the mid-point as a possible threshold. i.e,

$$S_{a <= x},\ S_{a >= y}$$

Continuous Attributes

- Example:

Temperature (T):	40	50	60	70	76	80	90
Class:	-	-	+	-	+	+	-

- Check thresholds: T < 55 or T < 65 or T<73 or T < 85

- Use the threshold with the highest information gain

Missing Values with Decision Trees

- Diagnosis = < temperature, blood_pressure,…, blood_test=?,…>

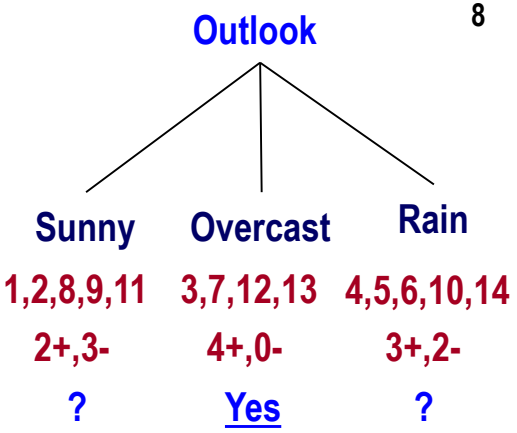
- Many times values are not available for some attributes during training or testing (e.g., medical diagnosis)

- Training: evaluate *Gain(S,a)* where in some of the examples a value for *a* is not given

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	???	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes

Missing Values At Training

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
8	Sunny	Mild	???	Weak	No



$Gain(S_{sunny}, Temp) =$

$Gain(S_{sunny}, Humidity) =$

Use:

A) the most common Humidity at Sunny

B) as (A) but with PlayTennis = No (useless in testing)

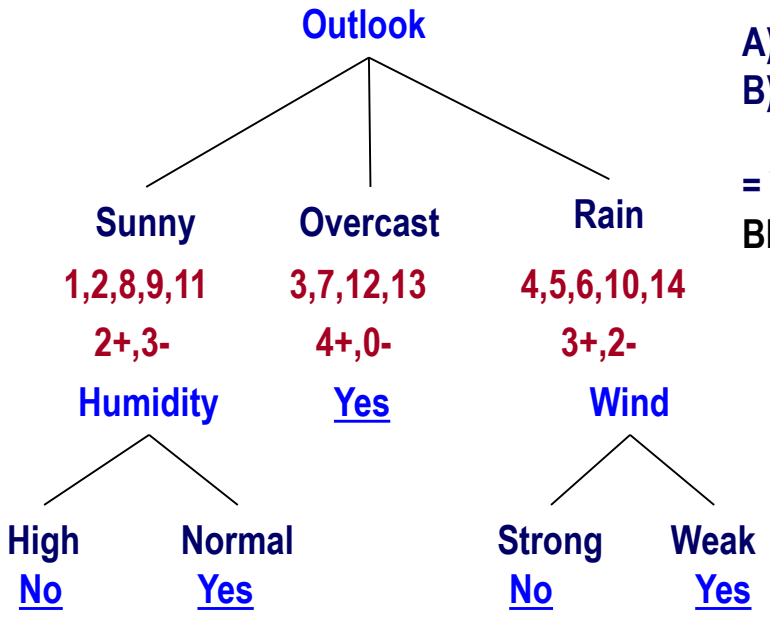
C) Fractional instances

Missing Values

- Diagnosis = < temperature, blood_pressure,…, blood_test=?,…>
- Many times values are not available for all attributes during training or testing (e.g., medical diagnosis)
- Training: evaluate *Gain(S,a)* where in some of the examples a value for *a* is not given
- Testing: classify an example without knowing the value of *a*

Missing Values At Testing

Outlook = ???, Temp = Hot, Humidity = Normal, Wind = Strong, label = ??



A) use most common value in training

B) Blend by labels:

5/14 Yes + 4/14 Yes +5/14 No

= Yes

Blend by probability

(estimated by counts)

Other Issues

- Attributes with different costs

-Solution: Change information gain so that low cost attributes are preferred

- Alternative measures for selecting attributes

When different attributes have different number of values
information gain tends to prefer those with many values (e.g. Date)
They do not generalize well

-Solution: Use GainRatio

A decision tree to distinguish homes in New York from homes in San Francisco

http://r2d3.us/visual-intro-to-machine-learning-part-1/

Summary: Decision Trees

- What is a decision tree?
- How to learn a decision tree from data?
  - Top-down induction to minimize classification error
  - Choose features using Information Gain
  - Continuous Attributes
  - Missing Values